

소프트웨어공학

소프트웨어 공학의 발전적 추세 2

1 CASE의 개념

2 CASE의 분류

3 정보저장소

CASE: Computer Aided Software Engineering

소프트웨어 개발 과정에서 사용되는 요구분석, 설계, 구현, 검사 및 디버깅 과정 전체 또는 일부를 컴퓨터와 전용 소프트웨어 도구를 사용하여 자동화 하는 것이다.

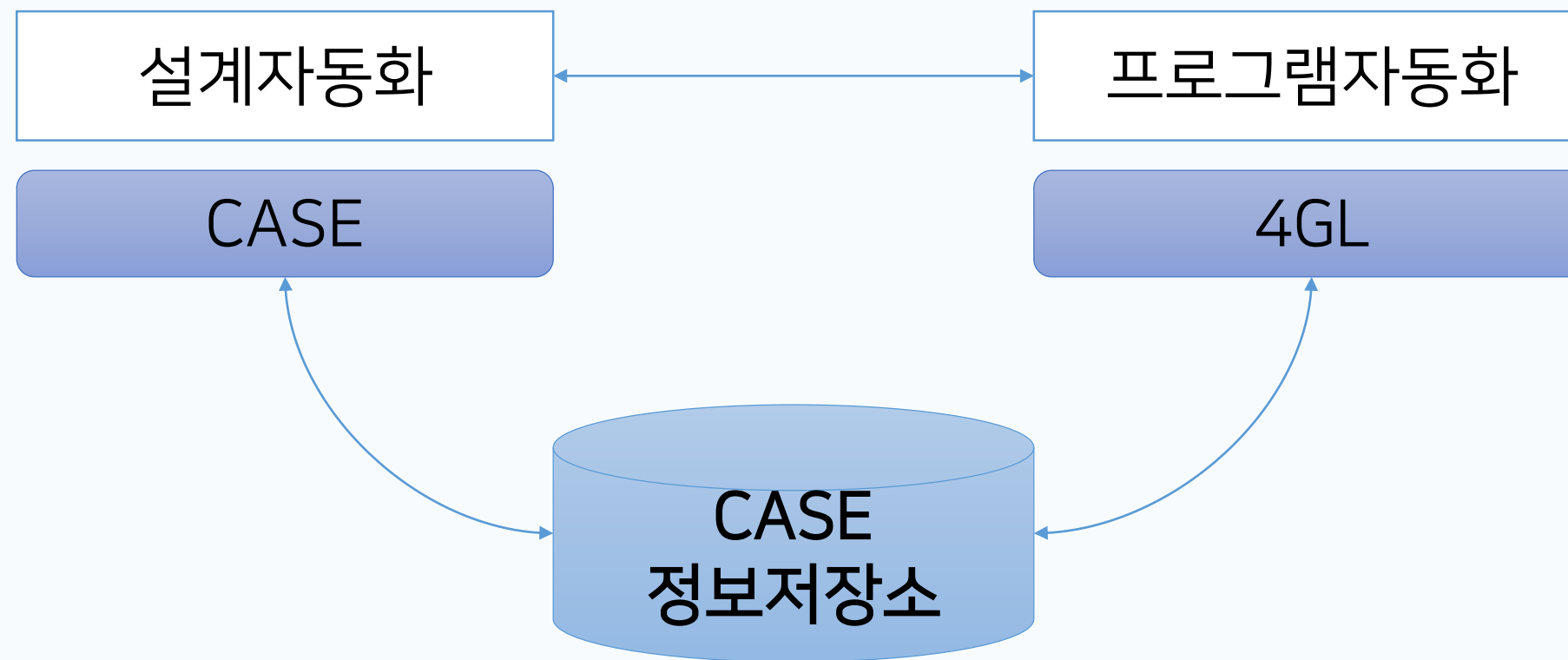
- 개요

- 소프트웨어 생명주기의 전체 단계를 연결해 주고 자동화해 주는 통합된 도구를 제공해 주는 기술이다.
- 소프트웨어 개발 도구와 방법론이 결합된 것으로, 정형화된 구조 및 방법을 소프트웨어 개발에 적용하여 생산성 향상을 구현하는 공학 기법이다.
- 소프트웨어 생명주기 모든 단계에 걸쳐 일관된 방법론을 제공하는 자동화 도구 (case tool) 들을 지원하고, 개발자들은 이 도구를 사용하여 소프트웨어 개발의 표준화를 지향하며, 자동화의 이점을 얻을 수 있게 해준다.

01

CASE의 개념

- 개요
 - 전체 소프트웨어 생산성과 신뢰성 문제에 중점을 두고 있다.
 - 개발 도구와 개발 방법론이 결합된것이다.



- CASE 등장 배경
 - 소프트웨어의 생산성을 향상시켜 소프트웨어의 위기를 해결하기 위해 사용하기 위함이다.
 - 소프트웨어공학적인 접근 방법으로 원리를 적용하고 체계적으로 작업하여 개발 및 유지보수 업무를 자동화하기 위함이다.
 - 컴퓨터 분야의 기술 발전으로 인해 크고 복잡한 소프트웨어 개발 요구가 증가함에 따라 체계적으로 개발할 수 있는 자동화된 도구를 도입하였다.

- CASE 사용의 이점

- 소프트웨어 개발 기간을 단축하고 개발 비용을 절감할 수 있다.
- 자동화된 기법을 통해 소프트웨어 품질이 향상된다.
- 소프트웨어의 유지보수를 간편하게 수행할 수 있다.
- 소프트웨어 생산성이 향상되고 생산, 운용 활동을 효과적으로 관리·통제할 수 있다.
- 품질과 일관성을 효과적으로 관리·통제할 수 있다.
- 품질과 일관성을 효과적으로 제어할 수 있다.
- 소프트웨어 개발의 모든 단계에 걸친 표준을 확립할 수 있다.
- 소프트웨어 모듈의 재사용성이 향상된다.
- 소프트웨어 개발 기법을 실용화할 수 있고, 문서화를 쉽게 작성할 수 있다.

- CASE 분류
 - CASE는 소프트웨어 생명주기의 어느 부분을 지원하느냐에 따라 계층적으로 분류할 수 있다.
 - 상위(upper) CASE
 - 소프트웨어 생명주기의 전반부에서 사용되는 것으로, 문제를 기술(description)하고 계획하며 요구 분석과 설계단계를 지원하는 CASE이다.
 - 여러 가지 명세와 문서를 작성하는 데 사용된다.
 - 상위 CASE 도구(tool)에는 SREM, PSL/PSA, SERA, FOUNDATION 등이 있다.

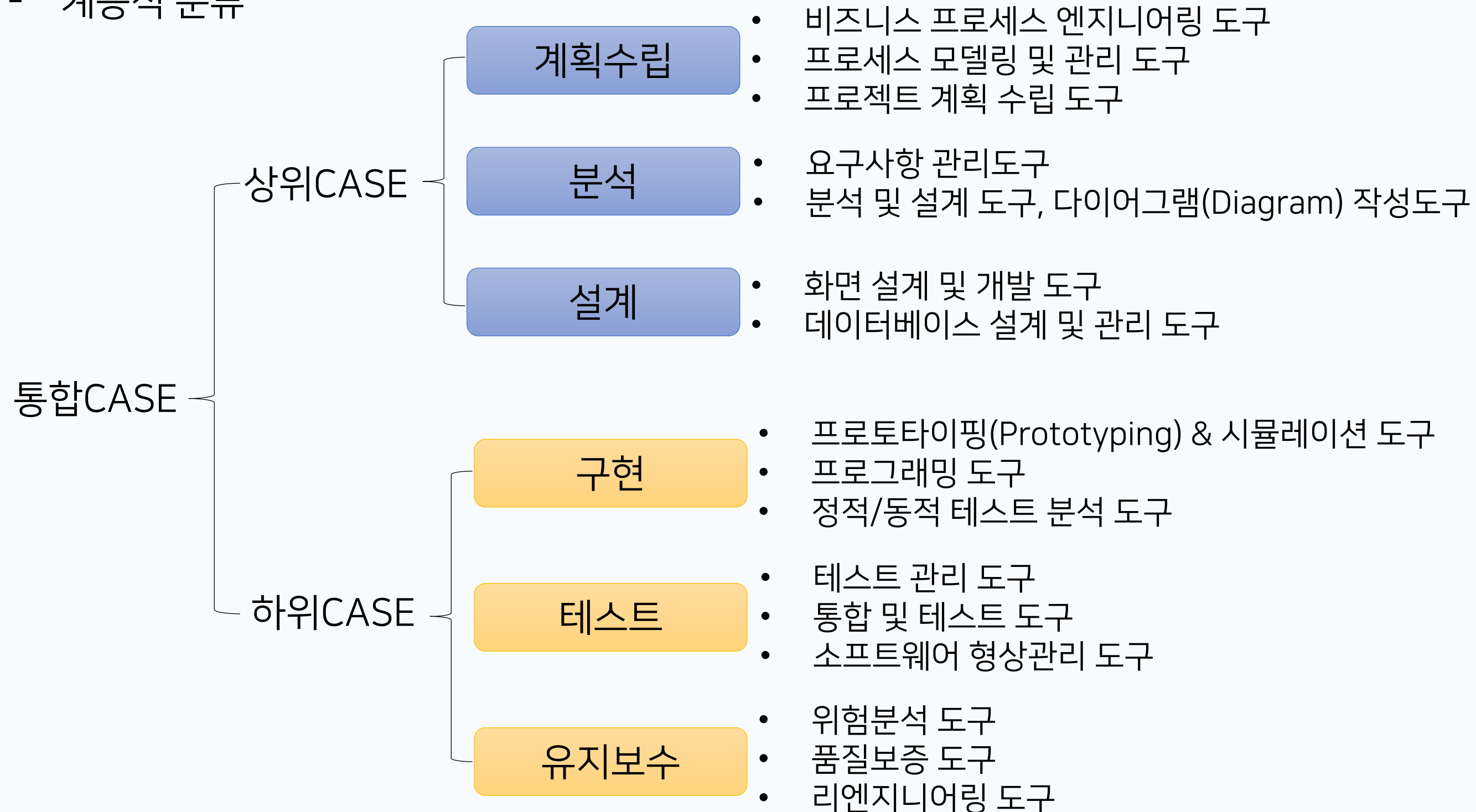
- CASE 분류
 - 하위(lower) CASE
 - 소프트웨어 생명주기의 하반부에서 사용되는 것으로 코드의 작성과 테스트, 문서화하는 과정을 지원하는 CASE이다.
 - 하위 CASE 도구에는 구문 중심 편집기, 코드 생성기 등이 있다.
 - 통합(intergrate) CASE
 - 소프트웨어 생명주기에 포함되는 전체 과정을 지원하기 위한 CASE로, 공통의 정보 저장 장소와 통일된 사용자 인터페이스를 사용하여 도구들을 통합한다.
 - 통합 CASE 도구에는 IEF, POWERTOOLS, TAGS/IOPL, TEAMWORK 등이 있다.

02

CASE의 분류

- CASE 분류

- 계층적 분류



02

CASE의 분류

- CASE 시스템 구성요소



- CASE 도구의 분류
 - 도구함(Toolkit)
 - SDLC의 한 단계나 특정 소프트웨어 작업 분야를 자동화하기 위한 통합된 도구의 집합
 - 워크벤치(Workbench)
 - SDLC 전체 작업들을 자동화하기 위한 통합된 도구의 결합
 - 개발방법론 지원도구(Methodology companion)
 - 특정 소프트웨어 개발 방법론의 작업들을 자동화시키기 위한 통합된 도구의 집합

- CASE 도구의 실패원인
 - 개발 조직의 기술 수준이 취약하다.
 - CASE에 대한 이해의 부족
 - 새로운 개발 환경에 대한 거부감
 - 목적에 부합하지 않는 CASE 도구 구입/사용
 - 경영, 관리의 중요성 무시

- 전통적인 소프트웨어 개발과 CASE 소프트웨어 개발의 비교

전통적인 소프트웨어 개발	CASE 소프트웨어 개발
코딩과 테스트를 강조 제안서에 의존한 분석 수작업 코딩 수작업 문서화 소프트웨어만 테스트 코딩의 유지보수	분석과 설계 강조 프로토타입핑의 반복으로 빠른 분석 자동 코드 생성기 이용 자동 문서화 생성기 이용 자동 설계 검증/확인기 이용 설계와 명세서도 유지보수

정보저장소
(repository)

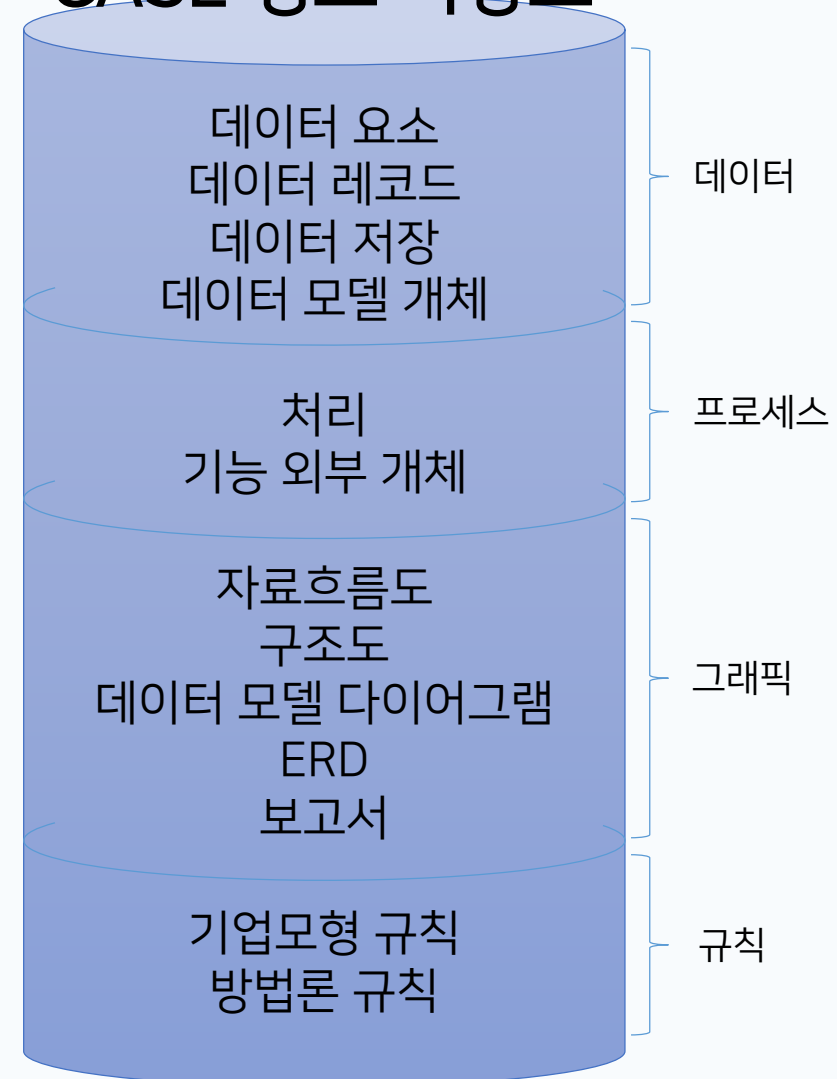
소프트웨어를 개발하는 과정 동안에 모아진 정보를 보관하여 관리하는 곳으로 CASE 정보 저장소, CASE 데이터베이스, 요구사항 사전, 저장소라고도 한다.

- 초기의 소프트웨어 개발 환경에서는 사람이 정보 저장소 역할을 하였지만 오늘날에는 데이터베이스가 정보 저장소 역할을 담당한다.
- 도구들의 통합, 소프트웨어 시스템의 표준화, 소프트웨어 시스템 정보의 공유, 소프트웨어 재사용성의 기본이 된다.
- SDLC의 한 단계에서 사용되는 CASE 도구들이 자료를 다른 단계로 전송할 수 있도록 도와주는 생명주기 통합기이다.

- 정보저장소의 내용과 기능

내용	기능
CASE 도구의 통합 시스템 명세서의 일관성 및 통합 제어 시스템 정보의 공유 문서의 표준화 시스템 문서 생성 코드 생성 소프트웨어 재사용 프로젝트 관리	보안 관리 엑세스 특권 버전 관리 다중 사용자 접근 변경 관리

CASE 정보 저장소



- 정보저장소의 역할
 - 자료 무결성
 - 정보 저장소에 대한 입력을 집중하고 관련된 객체 간의 일관성을 보장한다.
 - 정보 공유
 - 여러 개발자들과 여러 도구들 간에 정보를 공유할 수 있는 메커니즘을 제공한다.
 - 자료-도구 통합
 - 모든 도구들이 접근할 수 있는 자료 모형을 설정하고 자료에 대한 접근을 제어한다.

- 정보저장소의 역할
 - 자료-자료 통합
 - 다른 기능들이 성취될 수 있도록 자료 객체에 대한 관련된 DBMS를 설정한다.
 - 방법론 강제
 - 정보 저장소에 저장되어 있는 자료의 개체관계도가 의미하는 지정된 패러다임을 설정한다.
 - 문서 표준화
 - 소프트웨어 공학 문서의 생성을 위한 표준화된 접근 방법을 유도한다.

- 정보저장소의 이점

- 도구들과 생명주기 활동, 사용자들, 응용 소프트웨어들 사이의 통신과 소프트웨어 시스템의 정보 공유를 향상시킨다.
- 소프트웨어 시스템 구성 요소들과 시스템 정보가 정보 저장소에 의해 관리되므로 유지 보수성이 향상된다.
- CASE 도구들 간에 정보를 쉽게 교환하고, 사용자가 쉽게 새로운 도구를 추가할 수 있도록 해준다.
- CASE 도구들을 통합하여 통합 CASE 도구 사용을 가능하게 한다.
- 중복된 공통 정보를 통합하며 불필요한 정보를 제거한다.
- 생명주기 정보를 재사용할 수 있도록 한다.
- 소프트웨어 시스템의 이식과 변환을 용이하게 한다.